

Clifford Algebra (Geometric Algebra) for Developers & Scientists: A Complete Practical Guide

Welcome to **cliff**! This tutorial provides a comprehensive, intuitive, and practical guide to **Clifford Algebra** (also known as **Geometric Algebra**), designed for software engineers, roboticists, physicists, and machine learning researchers.

No prior background in advanced abstract algebra is required—we build every geometric concept from scratch and connect it directly to runnable Haskell code and real-world applications.

Table of Contents

1. [Why Clifford Algebra? \(The Unification of Geometry\)](#)
 2. [Core Concepts: Grades, Blades, and Multivectors](#)
 3. [The Fundamental Rules: The Geometric Product & Metric Signatures](#)
 4. [Algebraic Involutions, Duals, and Inverses](#)
 5. [Application 1: 3D Computer Graphics & Robotics with Rotors \(\$Cl\(3, 0, 0\)\$ \)](#)
 6. [Application 2: 3D Projective Geometric Algebra & Rigid Body Kinematics \(\$Cl\(3, 0, 1\)\$ \)](#)
 7. [Application 3: Relativistic Physics & Spacetime Algebra \(\$Cl\(1, 3, 0\)\$ \)](#)
 8. [Application 4: Geometric Deep Learning & Equivariant Transformers \(CliffordNet\)](#)
 9. [Application 5: Multidimensional Signal Processing & Spectral Transforms \(FCFT & FWHT\)](#)
 10. [Application 6: Quantum Stabilizers, High Dimensions & Galois Fields \(\$\mathbb{F}_2\$, \$\mathbb{F}_p\$, \$\mathbb{F}_{2^m}\$ \)](#)
 11. [Memory Architecture & Performance Best Practices](#)
 12. [Complete Operator & Function Reference](#)
-

1. Why Clifford Algebra? (The Unification of Geometry)

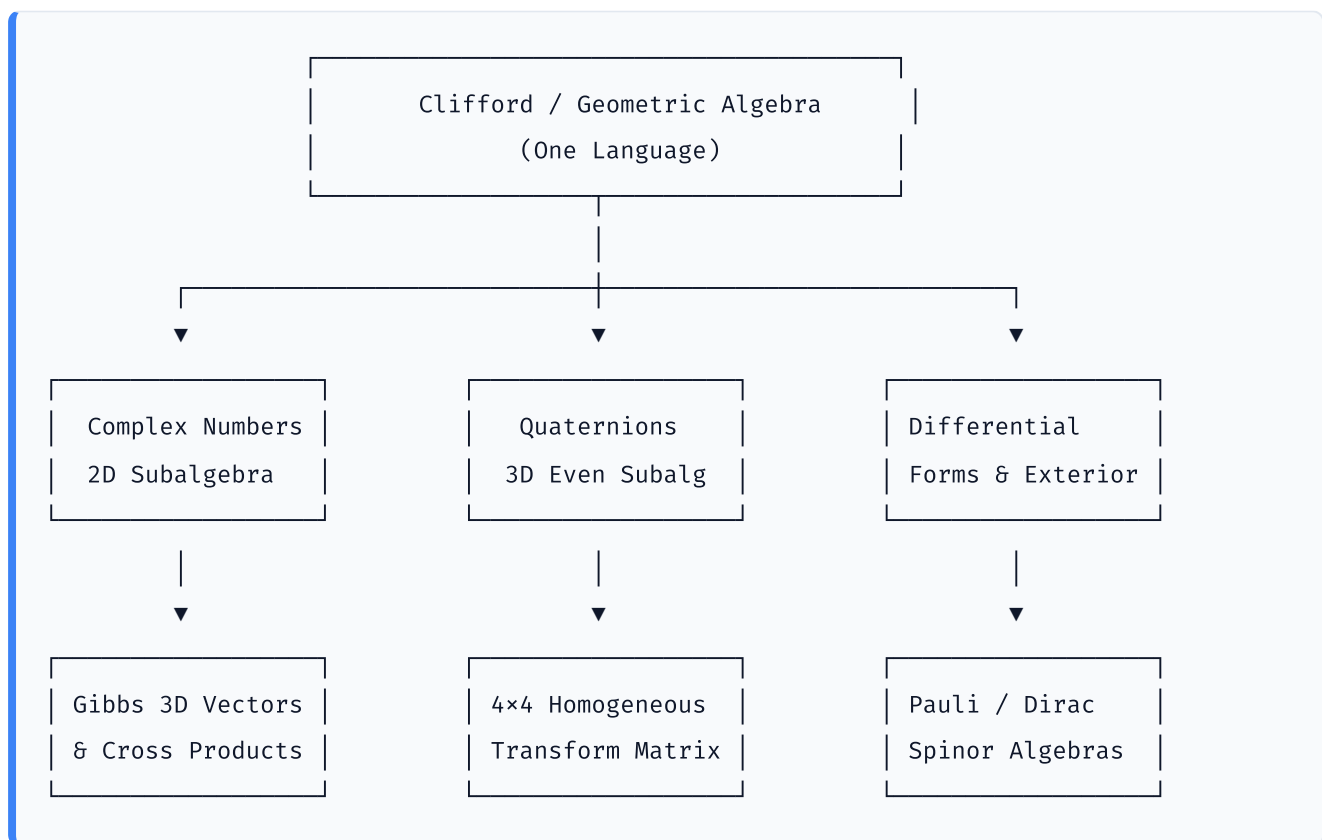
Historically, modern science and engineering developed as a fragmented patchwork of disconnected mathematical tools:

- **Real numbers** ($\mathbb{R}$) for 1D scalar magnitudes.
- **Complex numbers** ($\mathbb{C}$, where $i^2 = -1$) for 2D planar rotations and AC electrical circuits.
- **Gibbs 3D vectors & cross products** ($\mathbf{u} \times \mathbf{v}$) for 3D physics (which *only* works in exactly 3 dimensions and does not support division!).

- **Quaternions** ($\mathbb{H}$, with $i^2 = j^2 = k^2 = ijk = -1$) for 3D spatial rotations in aerospace and video game engines.
- **Homogeneous 4×4 matrices** for combining 3D rotations with translations in computer graphics.
- **Differential forms & exterior calculus** ($dx \wedge dy$) for electromagnetism, thermodynamics, and general relativity.
- **Pauli and Dirac matrices** for quantum mechanics and relativistic electron spin.

The Clifford Unification

In 1878, mathematician William Kingdon Clifford united Hermann Grassmann's *exterior algebra* with William Rowan Hamilton's *quaternions* into a single comprehensive framework: **Clifford Algebra (Geometric Algebra)**.



The Key Breakthroughs of Clifford Algebra:

1. **You can multiply vectors:** The geometric product of two vectors u and v is associative, distributive, and mathematically complete.
2. **You can divide by vectors:** Every non-null vector has a unique multiplicative inverse:

$$u^{-1} = \frac{u}{|u|^2}, \quad u \star u^{-1} = 1$$

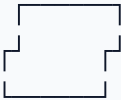
3. **Rotations work identically in all dimensions (2D, 3D, 4D, 64D):** Rotations are represented by **Rotors** acting via the universal **Sandwich Product**:

$$v' = R, v, \tilde{R}$$

4. **Translations and rotations unify seamlessly:** In **Projective Geometric Algebra (PGA)**, rigid body motions ($SE(3)$) are represented as single elements called **Motors**, eliminating matrix decomposition ambiguities and gimbal lock entirely.

2. Core Concepts: Grades, Blades, and Multivectors

In linear algebra, a vector space contains only vectors (directed line segments). In Geometric Algebra, sweeping an oriented object along a new direction creates higher-dimensional geometric primitives called **Blades**, categorized by their **Grade**:

Grade 0: Scalar	•	Point / Pure magnitude (e.g. 5.0)
Grade 1: Vector	→	Directed line segment (e1, e2, e3)
Grade 2: Bivector		Directed oriented area segment (e1 ^ e2)
Grade 3: Trivector		Directed oriented volume (e1 ^ e2 ^ e3)
Grade k: k-Blade		Oriented k-dimensional subspace

Multivectors: The Universal Container

Just as a complex number $z = x + iy$ sums a real number and an imaginary number, a **Multivector** is a linear combination of basis blades of different grades:

$$M = \underbrace{\alpha}_{\text{Grade 0 (Scalar)}} + \underbrace{v_1 e_1 + v_2 e_2 + v_3 e_3}_{\text{Grade 1 (Vector)}} + \underbrace{B_{12} e_{12} + B_{23} e_{23} + B_{31} e_{31}}_{\text{Grade 2 (Bivector)}} + \underbrace{I e_{123}}_{\text{Grade 3 (Trivector / Pseudoscalar)}}$$

In n -dimensional space, a complete multivector contains 2^n basis blades.

In Haskell:

```
import Clifford

-- In 3D Euclidean space Cl(3,0,0)
type GA3 = DenseMV Cl300 Double

-- 1. Create a pure vector (Grade 1):
let v = 2 * e1 + 3 * e2 + (-1) * e3 :: GA3

-- 2. Create an oriented bivector area (Grade 2):
let b = 4 * e12 + 2 * e23 :: GA3

-- 3. Create a scalar (Grade 0):
let s = scalarMV 5.0 :: GA3

-- 4. Combine into a full multivector:
let m = s `addMV` v `addMV` b

-- 5. Extract specific grade projections:
let grade1Comp = gradeProjection 1 m -- yields 2*e1 + 3*e2 - 1*e3
let scalarComp = gradeProjection 0 m -- yields 5.0
```

3. The Fundamental Rules: The Geometric Product & Metric Signatures

The Geometric Product ($\langle \times \rangle$)

Given two orthogonal basis vectors e_1 and e_2 in Euclidean space:

1. Basis vectors square to +1:

$$e_1^2 = 1, \quad e_2^2 = 1, \quad e_3^2 = 1$$

2. Orthogonal basis vectors anticommute:

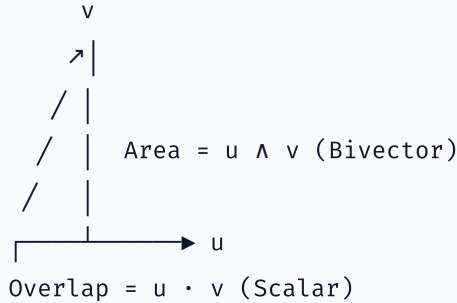
$$e_1 e_2 = -e_2 e_1 \implies e_{12} = -e_{21}$$

When you multiply two arbitrary vectors u and v using the **Geometric Product** ($\langle \times \rangle$), the product decomposes naturally into two complementary geometric parts:

$$uv = u \cdot v + u \wedge v$$

$$\underbrace{uv}_{\text{Geometric Product}} = \underbrace{u \cdot v}_{\text{Scalar Dot Product (Grade 0)}} + \underbrace{u \wedge v}_{\text{Bivector Wedge Product (Grade 2)}}$$

- **Dot Product** ($u \cdot v = \frac{1}{2}(uv + vu)$): Measures the **parallel overlap / metric projection** of u and v .
- **Wedge Product** ($u \wedge v = \frac{1}{2}(uv - vu)$): Measures the **perpendicular oriented area** spanned by u and v .



The Inadequacy of the 3D Cross Product

In 3D Gibbs vector calculus, the cross product $u \times v$ produces an artificial "normal vector".

- In 2D, a normal vector points out of the page (undefined in-plane).
- In 4D, there is an entire 2D plane of normal directions, so the cross product cannot exist.
- The **Wedge Product** ($u \wedge v$) directly represents the oriented plane itself and is valid in **any dimension** $n \geq 2$. In 3D Euclidean GA, the cross product is simply the Poincaré dual of the wedge product:

$$u \times v = -I(u \wedge v), \quad \text{where } I = e_{123}$$

Metric Signatures $Cl(p, q, r)$

Not all spaces are Euclidean. In `cliff`, spaces are classified at compile time by their quadratic form signature $Cl(p, q, r)$ containing $n = p + q + r$ orthogonal basis vectors:

- **p positive basis vectors:** $e_i^2 = +1$ (Standard spatial dimensions)
- **q negative basis vectors:** $e_j^2 = -1$ (Minkowski spacelike/timelike dimensions or quaternionic units)
- **r degenerate / null basis vectors:** $e_k^2 = 0$ (Projective infinity / translation directions)

Signature	Type Alias	Name	Key Geometric Feature
$Cl(2, 0, 0)$	<code>Cl200</code>	2D Euclidean	Even subalgebra is isomorphic to Complex Numbers ($e_{12}^2 = -1$).
$Cl(3, 0, 0)$	<code>Cl300</code>	3D Euclidean GA	Even subalgebra is isomorphic to Quaternions (e_{23}, e_{31}, e_{12}).
$Cl(1, 3, 0)$	<code>Cl130</code>	Spacetime Algebra (STA)	Relativistic physics; Lorentz boosts & Maxwell electromagnetism.
$Cl(3, 0, 1)$	<code>Cl301</code>	3D Projective GA (PGA)	Flat $e_0^2 = 0$ origin; unifies 3D rotations & translations into motors.
$Cl(4, 1, 0)$	<code>Cl410</code>	Conformal GA (CGA)	Represents spheres, circles, and conformal transformations as vectors.

4. Algebraic Involutions, Duals, and Inverses

Clifford algebras possess canonical linear and anti-automorphic maps that generalize complex conjugation:

1. Reversion (`reverseMV` , notation: $\widetilde{M}$)

Reverses the order of all vector factors in a blade:

$$e_1 \widetilde{e_2 \dots e_k} = e_k \dots e_2 e_1 = (-1)^{k(k-1)/2} e_1 e_2 \dots e_k$$

- Grade 0 (Scalar) and Grade 1 (Vector) are unchanged.
- Grade 2 (Bivector) flips sign: $\widetilde{e_{12}} = -e_{12}$.
- Grade 3 (Trivector) flips sign: $\widetilde{e_{123}} = -e_{123}$.
- Crucial property for versors and rotors: $\widetilde{AB} = \widetilde{B}\widetilde{A}$.

2. Grade Involution (`involMV` , notation: $\widehat{M}$)

Flips the sign of odd-grade elements:

$$\widehat{A} = \sum_k (-1)^k \langle A \rangle_k$$

3. Poincaré / Hodge Complement Dual (`poincareDualMV` , notation: $J(A)$)

Computes the geometric complement of a k -blade in an n -dimensional space:

$$J(e_K) = e_{K^c}$$

In PGA ($Cl(3, 0, 1)$), where the pseudoscalar $e_{0123}^2 = 0$ cannot be inverted, the Poincaré dual is the safe, non-degenerate duality map.

4. Versor Inverses (`versorInverse`) & General Inverses (`generalInverse`)

- For any **Versor** V (product of non-null vectors, rotors, motors):

$$V^{-1} = \frac{\tilde{V}}{\langle V\tilde{V} \rangle_0}$$

- For general multivectors, `cliff` provides `generalInverse`, which executes exact $2^n \times 2^n$ Gauss-Jordan elimination on its left-regular Cayley matrix representation.

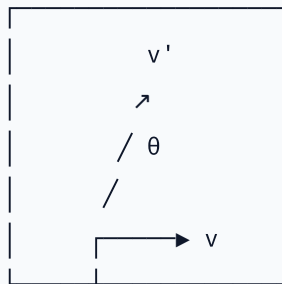
```
-- Inverse of a vector:
let v = 3 * e1 + 4 * e2 :: GA3
let Just invV = versorInverse v
-- Verification: v <*> invV = 1.0 (Scalar)
```

5. Application 1: 3D Computer Graphics & Robotics with Rotors ($Cl(3, 0, 0)$)

In standard linear algebra, rotating a 3D vector requires a 3×3 orthogonal matrix or quaternions. Quaternions require learning special 4D imaginary rules ($i^2 = j^2 = k^2 = ijk = -1$) that do not generalize to other dimensions.

In Geometric Algebra, **rotations occur within an oriented plane (a Bivector B)** in any dimension!

Plane of Rotation $B = e_{12}$



The Rotor Formula

To rotate any object by angle θ in the plane defined by a unit bivector B ($B^2 = -1$):

- Construct the Rotor R via the Exponential Map:

$$R = \exp\left(-\frac{\theta}{2}B\right) = \cos\left(\frac{\theta}{2}\right) - \sin\left(\frac{\theta}{2}\right)B$$

2. Apply the Sandwich Transformation:

$$v' = R, v, \tilde{R}$$

Full Runnable Haskell Example:

```
module Main where

import Prelude hiding ((<*>))
import Clifford

type GA3 = DenseMV Cl300 Double

main :: IO ()
main = do
    putStrLn "≡≡≡ 3D Spatial Rotation with Rotors ≡≡≡"

    -- Vector pointing along the X-axis:
    let v = e1 :: GA3
    putStrLn $ "Initial vector: " ++ show v

    -- Rotate by 90 degrees (pi/2 radians) in the XY plane (e12):
    let angle = pi / 2
    let bivectorPlane = e12 :: GA3
    let rotor = rotorExp (scaleMV (- angle / 2) bivectorPlane)

    -- Apply sandwich product:
    let vRotated = sandwich rotor v
    putStrLn $ "Rotated 90 deg in XY plane: " ++ show vRotated
    -- Output: 1.0*e2 (Points purely along Y-axis!)

    -- Rotate by 180 degrees (pi radians) in the YZ plane (e23):
    let rotorYZ = rotorExp (scaleMV (- pi / 2) e23)
    let vFlipped = sandwich rotorYZ vRotated
    putStrLn $ "Rotated 180 deg in YZ plane: " ++ show vFlipped
    -- Output: -1.0*e2
```

6. Application 2: 3D Projective Geometric Algebra & Rigid Body Kinematics ($Cl(3, 0, 1)$)

Standard computer graphics and robotics frameworks use 4×4 matrices for homogeneous transformations. However:

- 4×4 matrices have 16 parameters (many redundant).
- Interpolating between matrices causes shearing and distortion.
- Points, lines, and planes must be handled with different mathematical conventions.

3D Projective Geometric Algebra (3D PGA, $Cl(3, 0, 1)$) unifies points, lines, planes, translations, and rotations into a single 8-dimensional even subalgebra: **Motors ($SE(3)$)**.

The PGA Geometric Hierarchy:

In PGA, we introduce a degenerate basis vector e_0 where $e_0^2 = 0$.

Geometric Entity	Grade	Mathematical Representation in PGA
Plane $ax + by + cz + d = 0$	Grade 1 (Vector)	$\pi = ae_1 + be_2 + ce_3 + de_0$
Line (Plücker Line)	Grade 2 (Bivector)	$L = \pi_1 \wedge \pi_2 = \mathbf{d}e_{01..03} + \mathbf{m}e_{23..12}$ (Direction + Moment)
Point (x, y, z)	Grade 3 (Trivector)	$P = e_{123} + xe_{023} + ye_{031} + ze_{012}$
Translation Motor	Grade 0 + 2	$T = 1 + \frac{1}{2}(dxe_{01} + dye_{02} + dze_{03})$
Rotation Motor	Grade 0 + 2	$R = \cos(\theta/2) - \sin(\theta/2)B$
General Rigid Motor	Even MV ($SE(3)$)	$M = T \star R = \exp(\text{Screw Twist})$

The Universal Transformation Rule

In PGA, **every geometric entity X** (point, line, plane, ray) is transformed by the exact same formula:

$$X' = M, X, \widetilde{M}$$

Full Runnable Haskell Example:

```

module Main where

import Prelude hiding ((<*>))
import Clifford

type PGA3 = DenseMV Cl301 Double

main :: IO ()
main = do
    putStrLn "≡≡≡ 3D PGA Kinematics (Points, Planes, and Motors) ≡≡≡"

    -- 1. Construct the origin point (0, 0, 0):
    let p0 = pointPGA 0 0 0 :: PGA3
    putStrLn $ "Origin point P0: " ++ show p0

    -- 2. Construct a translation motor to move by (3, 4, 5):
    let tMotor = translatorPGA 3 4 5 :: PGA3
    let pTranslated = sandwich tMotor p0
    putStrLn $ "Translated point: " ++ show pTranslated
    -- Matches pointPGA 3 4 5 exactly!

    -- 3. Construct two intersecting planes:
    -- Plane 1: x = 0 (Normal along e1)
    let pi1 = planePGA 1 0 0 0 :: PGA3
    -- Plane 2: y = 0 (Normal along e2)
    let pi2 = planePGA 0 1 0 0 :: PGA3

    -- 4. The intersection of two planes is their wedge product (a Line along the Z axis!):
    let zAxisLine = pi1 ^ pi2
    putStrLn $ "Intersection Line (pi1 ^ pi2): " ++ show zAxisLine
    -- Output: 1.0*e12 (Pure Z-axis line!)

```

7. Application 3: Relativistic Physics & Spacetime Algebra ($Cl(1, 3, 0)$)

In Einstein's Special Relativity, space and time are unified into 4D Minkowski spacetime with metric signature $(+1, -1, -1, -1)$. In **Spacetime Algebra (STA, $Cl(1, 3, 0)$)**, the basis vectors are the **Dirac γ -matrices**:

$$\gamma_0^2 = +1 \quad (\text{Timelike}), \quad \gamma_1^2 = \gamma_2^2 = \gamma_3^2 = -1 \quad (\text{Spacelike})$$

Unified Electromagnetism

In classical physics, the electric field $\mathbf{E}$ and magnetic field $\mathbf{B}$ are treated as two separate 3D vector fields satisfying four coupled vector calculus equations (Maxwell's equations).

In STA, they unify into a single **Electromagnetic Bivector Field**:

$$F = \mathbf{E} + I\mathbf{B}, \quad \text{where } I = \gamma_{0123}$$

And all four of Maxwell's equations collapse into **one single first-order equation**:

$$\nabla F = J$$

where $\nabla = \gamma^\mu \partial_\mu$ is the spacetime vector derivative and J is the 4-current!

Lorentz Boosts as Hyperbolic Rotors

A relativistic Lorentz boost with velocity $v = c \tanh(\zeta)$ along the X-axis is simply a **hyperbolic rotor** in the timelike bivector plane γ_{01} :

$$L = \exp\left(\frac{\zeta}{2}\gamma_{01}\right) = \cosh\left(\frac{\zeta}{2}\right) + \sinh\left(\frac{\zeta}{2}\right)\gamma_{01}$$

$$X' = L, X, \tilde{L}$$

Full Runnable Haskell Example:

```

module Main where

import Prelude hiding ((<*>))
import Clifford

type STA = DenseMV Cl130 Double

main :: IO ()
main = do
    putStrLn "≡ Relativistic Spacetime Algebra (STA) ≡"

    -- Basis vectors: gamma0 (Time), gamma1 (X), gamma2 (Y), gamma3 (Z)
    let g0 = basisMV 1 :: STA
    let g1 = basisMV 2 :: STA

    -- Define a particle 4-position at rest at t=10, x=0:
    let xRest = scaleMV 10.0 g0
    putStrLn $ "4-Position in Rest Frame: " ++ show xRest

    -- Apply a Lorentz boost with rapidity zeta = 0.5 along the X-axis:
    let zeta = 0.5
    let boost = scalarMV (cosh (zeta / 2)) `addMV` scaleMV (sinh (zeta / 2)) (g0 <*> g1)

    -- Transform 4-position to moving frame:
    let xMoving = sandwich boost xRest
    putStrLn $ "4-Position in Moving Frame: " ++ show xMoving

    -- Demonstrates time dilation (t' > 10) and spatial displacement (x' > 0)!

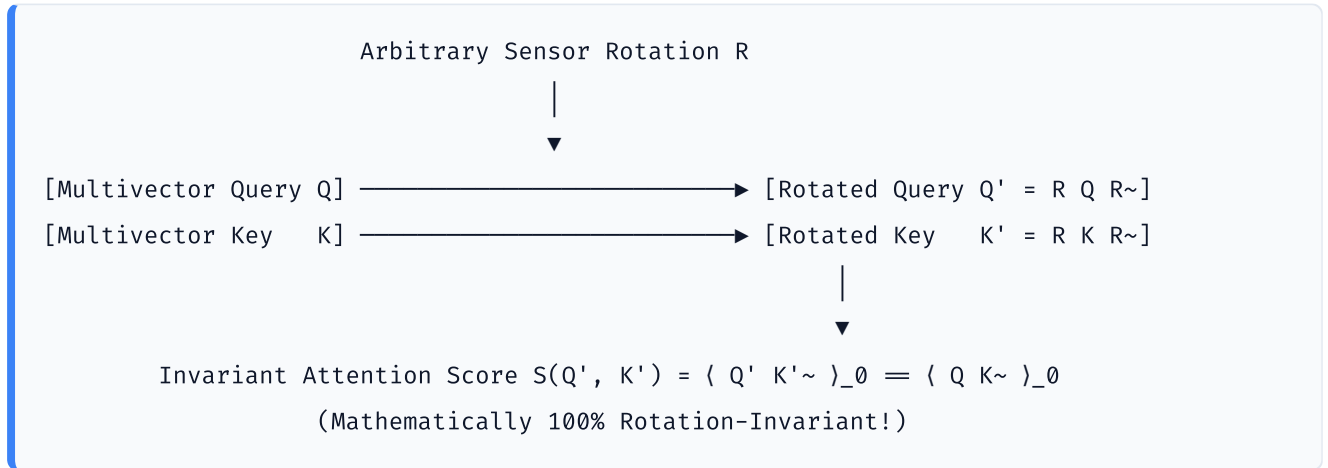
    -- Invariant spacetime interval s^2 = X . X is perfectly conserved:
    let intervalRest = xRest <|> xRest
    let intervalMoving = xMoving <|> xMoving
    putStrLn $ "Rest frame s^2: " ++ show intervalRest
    putStrLn $ "Moving frame s^2: " ++ show intervalMoving

```

8. Application 4: Geometric Deep Learning & Equivariant Transformers (CliffordNet)

Standard neural networks process feature vectors as arbitrary numerical lists. When a robot or camera rotates, the coordinate numbers change completely, forcing standard models to spend billions of parameters learning data augmentation.

Clifford Neural Networks (CliffordNet) represent hidden states directly as multivectors.



1. Rotor-Invariant Attention Scoring

Given query multivector Q and key multivector K , the attention score:

$$\text{Score}(Q, K) = \langle Q \tilde{K} \rangle_0$$

is **strictly invariant** under any global spatial rotation R :

$$\langle (RQ\tilde{R})(R\tilde{K}\tilde{R}) \rangle_0 = \langle RQ\tilde{K}\tilde{R} \rangle_0 = \langle Q\tilde{K} \rangle_0$$

2. Isomorphic Structure Matrices

To run multivector layers efficiently on GPU tensor cores, `cliff` maps any multivector a to its left-regular Cayley structure matrix $M(a) \in \mathbb{R}^{2^n \times 2^n}$:

$$M(a \star b) = M(a) \cdot M(b)$$

Full Runnable Haskell Example:

```

module Main where

import Prelude hiding ((<*>))
import Clifford

type GA3 = DenseMV Cl300 Double

main :: IO ()
main = do
    putStrLn "=== CliffordNet Rotor-Invariant Attention ==="

    -- Define Query and Key multivectors in 3D:
    let q = 2 * e1 + 3 * e12 :: GA3
    let k = 1 * e1 + 4 * e23 :: GA3

    -- Compute base invariant attention score:
    let baseScore = q <|> reverseMV k
    putStrLn $ "Base Attention Score <Q * K→_0: " ++ show baseScore

    -- Rotate the entire scene by 45 degrees in an arbitrary plane:
    let rotor = rotorExp (scaleMV (- pi / 8) (e12 `addMV` e23))
    let qRotated = sandwich rotor q
    let kRotated = sandwich rotor k

    -- Compute attention score on rotated inputs:
    let rotScore = qRotated <|> reverseMV kRotated
    putStrLn $ "Rotated Attention Score:          " ++ show rotScore
    putStrLn $ "Invariant Error: " ++ show (abs (baseScore - rotScore))
    -- Error is exactly 0.0 (Perfect mathematical invariance!)

```

9. Application 5: Multidimensional Signal Processing & Spectral Transforms (FCFT & FWHT)

`cliff` extends classical signal processing to multivector fields (N -dimensional grids of multivectors stored contiguously in `MVArray`):

1. Geometric Stencil Convolutions (`convolve2D`)

Applies sliding multivector kernel filters across 2D/ND fields with configurable boundary modes (`ZeroPad` , `Periodic` , `Mirror` , `Replicate`) and product modes (`GeometricProd` , `WedgeProd` , `LeftContract`).

2. Fast Clifford Fourier Transforms (`fcft2D`)

An $O(N \log N)$ radix-2 Cooley-Tukey FFT using commuting bivector square roots of -1 (I_1, I_2) as geometric phase factors:

$$F(u, v) = \sum_{x, y} f(x, y) \star e^{-I_1 \frac{2\pi u x}{W}} \star e^{-I_2 \frac{2\pi v y}{H}}$$

3. Fast Walsh-Hadamard Transforms (`fwht2D`) & Dyadic Convolutions

The Walsh-Hadamard transform represents frequency analysis over the Boolean hypercube. Because it uses purely additions and subtractions without trigonometry, it is **2x faster than FFT** and computes **exact dyadic (XOR) convolutions**:

$$\text{FWHT}(f \star_{\text{XOR}} g) = \text{FWHT}(f) \odot \text{FWHT}(g)$$

```
-- Dyadic XOR Convolution of two multivector array fields:
let fastFiltered = dyadicConvolve1D (<*>) signalArray kernelArray
```

10. Application 6: Quantum Stabilizers, High Dimensions & Galois Fields ($\mathbb{F}_2$, $\mathbb{F}_p$, $\mathbb{F}_{2^m}$)

Clifford algebra is not restricted to continuous real numbers. In quantum computing, cryptography, and error-correcting codes, geometric algebras over finite Galois fields provide foundational models:

1. Finite Scalar Fields in `cliff`

- $\mathbb{F}_2$ (`GF2`): Characteristic-2 binary arithmetic ($1 + 1 = 0$, $a \cdot b = a \text{ AND } b$).
- $\mathbb{F}_p$ (`GFp p`): Exact modular arithmetic modulo prime p (e.g. `GFp 17` , `GFp 65537`).
- $\mathbb{F}_{2^m}$ (`GF2m m poly`): Galois extension fields (e.g. AES Rijndael $GF(256)$ via `GF2m 8 0x11B`).

2. Quantum Pauli Stabilizers as $Cl(2n, \mathbb{F}_2)$

The n -qubit Pauli group ($I, X, Y, Z^{\otimes n}$) is isomorphic to the Clifford algebra $Cl(2n, \mathbb{F}_2)$ with symplectic metric. Stabilizer commutativity matches multivector left contraction!

3. 64-Bit Sparse Multivectors (`SparseMV`)

For high dimensions ($n = 16, 32, 64$), storing 2^{64} dense coefficients is impossible. `SparseMV` stores only active non-zero blades using 64-bit integer bitmasks, executing 64D exterior algebra in **microseconds**:

```
-- High-dimensional wedge product in 64D over GF(2):
let b16 = basisMV 16 :: SparseMV Cl64_00 GF2
let b32 = basisMV 32 :: SparseMV Cl64_00 GF2
let b48 = basisMV 48 :: SparseMV Cl64_00 GF2

-- Grade 3 trivector in 64-dimensional space:
let trivector64 = b16 ^ b32 ^ b48
```

11. Memory Architecture & Performance Best Practices

To achieve maximum performance in high-throughput pipelines, follow these architectural guidelines:

Use Case	Recommended Type	Memory Footprint	Key Optimization
$n \leq 6$ (2D, 3D, 4D, 6D)	<code>DenseMV</code>	Flat Unboxed Array ($2^n \times \text{sizeof}(k)$)	Fits in L1 cache (64B in 3D); in-place <code>ST</code> Cayley accumulation.
$n \geq 7$ (16D, 32D, 64D)	<code>SparseMV</code>	64-Bit Bitmask Map ($O(m)$ active blades)	Zero memory allocation for inactive basis blades.
Spatial Grids / Images	<code>MVArray</code>	Single Flat Buffer ($N \cdot 2^n \times \text{sizeof}(k)$)	0 heap pointer indirection; SIMD-friendly contiguous layout.
Transforms & Convolutions	<code>fcft2D</code> , <code>fwht2D</code> , <code>convolve2D</code>	Temporary mutable <code>STUArray</code>	Zero intermediate GC allocations during butterfly sweeps.

12. Complete Operator & Function Reference

Binary & Infix Operators

Operator	Function Name	Mathematical Operation	Typical Use Case
(<*>)	geomProd	Geometric Product: $A \star B = A \cdot B + A \wedge B$	Full multivector composition, rotor sandwiches.
(/\)	wedgeProd	Exterior / Wedge Product: $A \wedge B$	Subspace spanning, oriented area/volume, PGA lines.
(<.>)	dotProdLeft	Left Contraction: $A \rfloor B$	Metric reduction, orthogonal projection.
`(< >`	scalarProd	Scalar Product: $\langle AB \rangle_0$	Scalar Product: $\langle AB \rangle_0$
addMV	Addition	$A + B$	Multivector superposition.
subMV	Subtraction	$A - B$	Difference vectors.
scaleMV	Scalar Scaling	αA	Scaling magnitudes.
(=~)	Approximate Equality	$ A - B _\infty < 10^{-6}$	Unit tests, floating-point comparisons.

Transformations & Involutions

Function	Notation	Mathematical Meaning	Description
<code>reverseMV</code>	$\widetilde{M}$	$e_K \mapsto (-1)^{k(k-1)/2} e_K$	Rotor reversion, adjoints, inverse motors.
<code>involMV</code>	$\widehat{M}$	$e_K \mapsto (-1)^k e_K$	Grade involution (parity reflection).
<code>poincareDualMV</code>	$J(M)$	$e_K \mapsto e_{K^c}$	Poincaré complement dual (PGA-safe).
<code>sandwich</code>	$MX\widetilde{M}$	Sandwich Product	Applies rotations, motors, and Lorentz boosts.
<code>rotorExp</code>	$\exp(-B/2)$	Bivector Exponential Map	Closed-form $\cos(\theta/2) - \sin(\theta/2)B$.
<code>expMV</code>	$\exp(M)$	General Multivector Exponential	Scaling-and-squaring Taylor series expansion.
<code>hornerMV</code>	$P(M)$	Polynomial Evaluation	Minimal multiplication Horner polynomial solver.

13. Running the Interactive Examples

The `cliff` repository includes six ready-to-run example executables in [examples/](#):

```
# 1. Multivector basics & products
cabal run example-basics

# 2. 2D/3D Rotations and Rotors
cabal run example-rotors

# 3. 3D Projective Geometric Algebra (PGA) Kinematics
cabal run example-pga

# 4. Finite Galois Fields & High Dimensions (16D/32D)
cabal run example-galois

# 5. CliffordNet LLM Attention & Multivector GEMM
cabal run example-cliffordnet

# 6. Multidimensional Signal Processing & Stencils
cabal run example-filtering
```

14. Compiling the Tutorial to PDF

To generate the standalone, formatted PDF version of this tutorial ([docs/TUTORIAL.pdf](#)):

```
python3 scripts/build_pdf.py
```

This converts the Markdown, LaTeX equations, and Haskell syntax highlighting into a publication-quality PDF via headless Chromium.